A MINI-PROJECT REPORT ON

CHROMATIC VISION

Intelligent Color Detection and Object Tracking Using Image Processing Techniques

Submitted in partial fulfilment of the requirements for the course

IMAGE AND VIDEO ANALYTICS

of

BACHELOR OF TECHNOLOGY

Submitted by

Name: _____

Roll No.: _____

Under the guidance of

_____

(Designation)

DEPARTMENT OF _____

_____ (Name of the Institute)

Academic Year 20___ – 20___

CERTIFICATE

This is to certify that the mini-project report entitled **"Chromatic Vision – Intelligent Color Detection and Object Tracking Using Image Processing Techniques"** is a bonafide record of the work carried out by _____ (Roll No. _____) in partial fulfilment of the requirements for the course **Image and Video Analytics** of the Bachelor of Technology programme during the academic year 20___ – 20___.

Signature of the Guide
Name: _____

Signature of the Head of Department
Name: _____

Internal Examiner

External Examiner

Date: _____
Place: _____

ABSTRACT

Colour is one of the simplest and most reliable visual cues for locating objects in a scene. This project presents **Chromatic Vision**, a web-based application that detects objects of a chosen colour in images and videos, measures them, and tracks them across video frames using only classical image processing techniques.

Each image or video frame is converted from the BGR colour space to the HSV colour space. A binary mask of the target colour is produced with cv2.inRange(); red is handled with two hue ranges because it lies at both ends of the hue circle. The mask is cleaned using morphological opening and closing, and individual objects are identified through contour analysis, which gives a bounding box, area and centroid for every object. For videos, a centroid-based tracker links detections between consecutive frames and keeps object identities through short occlusions.

The application is developed in Python with OpenCV, NumPy and Streamlit, and provides interactive analytics using Plotly and Matplotlib. It was evaluated on a purpose-built synthetic dataset with exactly known ground truth. The detected object counts matched the ground truth for every colour tested, neutral (white, grey, black) objects were never falsely detected, and both tracked objects in the sample video kept their identities for all 200 frames, including a 9-frame occlusion. The average processing time was about 9–10 ms per 640×360 frame on a desktop CPU. The system was verified with 34 automated unit tests and 23 automated browser-based checks, all of which passed.

Keywords: *HSV colour space, colour segmentation, morphological operations, contour detection, centroid tracking, OpenCV, Streamlit.*

TABLE OF CONTENTS

1. INTRODUCTION 1
1.1 Project Overview 1
1.2 Scope of the Project 1
1.3 Organisation of the Report 2
2. PROBLEM STATEMENT AND OBJECTIVES 3
2.1 Problem Statement 3
2.2 Objectives 3
2.3 Limitations of Existing Approaches 4
3. METHODOLOGY 5
3.1 Overall Approach 5
3.2 Image Processing Techniques Used 5
3.3 HSV Colour Model and the Red Hue Problem 6
3.4 Algorithm 1 – Colour Object Detection 7
3.5 Algorithm 2 – Centroid-Based Tracking (for each frame t) 7
4. SYSTEM ARCHITECTURE 8
4.1 Architecture of the Processing Model 8
4.2 Processing Workflow 8
4.3 Technologies Used 9
4.4 Hardware and Software Requirements 9
5. DATASET DETAILS 10
5.1 Description of the Dataset 10
5.2 User Data 11
6. IMPLEMENTATION 12
6.1 Project Structure 12
6.2 Image Analysis Module 12
6.3 Video Tracking Module 13
6.4 User Interface 13
6.5 Error Handling and Resource Management 13
6.6 Testing Method 13
7. RESULTS AND DISCUSSION 15
7.1 Image Detection Results 15
7.2 Video Tracking Results 16
7.3 Summary of Test Cases 17
8. SCREENSHOTS 18
9. CONCLUSION 28
9.1 Conclusion 28
9.2 Limitations 28
9.3 Future Enhancements 28
REFERENCES 29

1. INTRODUCTION

1.1 Project Overview

Image and video analytics is concerned with automatically extracting meaningful information from visual data. Among all visual features, colour is one of the easiest to compute and one of the most useful in practice: fruit sorting, counting coloured components, following a ball in a sports clip or tracking a coloured marker on a robot all depend on it. Deep-learning detectors are powerful, but they need large labelled datasets, long training and usually a GPU, and their decisions are hard to explain. When the main distinguishing property of an object is its colour, classical colour segmentation is faster, fully transparent and requires no training.

Chromatic Vision packages this classical pipeline into an easy-to-use web application. The user uploads an image or a video (or loads a bundled sample), selects a target colour from a palette or defines a custom HSV range, and the system returns:

- a binary segmentation mask of the selected colour and the segmented image,
- the bounding box, area and centroid of every detected object, with the object count,
- for videos, a processed video in which each object carries a tracking ID and a motion trail,
- analytics charts computed from the actual results (colour distribution, object areas, detections per frame, processing time, trajectories),
- downloadable processed images, masks, videos and CSV data.

Every intermediate stage of the pipeline (HSV conversion, threshold mask, erosion, dilation, opening, closing and the final detections) is displayed in the application, so the project also works as a visual teaching aid for the topics of the Image and Video Analytics course.

1.2 Scope of the Project

- Colour-based detection of objects in still images (JPG, JPEG, PNG).
- Frame-by-frame colour detection and multi-object tracking in videos (MP4, AVI, MOV).
- Six predefined colours (red, orange, yellow, green, blue, purple) and a custom HSV range mode.
- Visual analytics, session statistics and export of results.

- Runs completely offline on a normal computer: no IoT hardware, cloud services, paid APIs or GPU.

1.3 Organisation of the Report

Chapter 2 states the problem and objectives. Chapter 3 describes the methodology and the image processing techniques used. Chapter 4 presents the system (model) architecture. Chapter 5 gives the dataset details. Chapter 6 explains the implementation, Chapter 7 presents the results, Chapter 8 contains the application screenshots and Chapter 9 concludes the report.

2. PROBLEM STATEMENT AND OBJECTIVES

2.1 Problem Statement

Manually locating, counting and following coloured objects in images and videos is slow, subjective and does not scale. Simple approaches that threshold the raw RGB values are unreliable, because a change in illumination (a shadow, a highlight or a darker scene) changes all three RGB channels together, so the same object may fall outside a fixed RGB threshold. Basic classroom scripts usually display a single OpenCV window, have no way to tune parameters, give no measurements or analytics, and crash on invalid input.

The problem addressed by this project is therefore: to design and implement a reliable, explainable and easy-to-use system that (i) segments a user-selected colour robustly under normal lighting variation, (ii) removes noise and fills holes in the segmented regions, (iii) separates the result into individual objects and measures each one, (iv) maintains a consistent identity for every object across the frames of a video, even when detections are briefly missed, and (v) presents the results and analytics through a professional graphical interface.

2.2 Objectives

1. To detect objects of a selected colour using HSV colour-space segmentation.
2. To handle the colour red correctly using two hue ranges.
3. To clean binary masks using erosion, dilation, morphological opening and closing.
4. To identify objects using contour detection, bounding boxes and centroid calculation.
5. To process videos frame by frame and track moving objects using centroid-based tracking.
6. To provide visual analytics computed only from real processing results.
7. To allow the user to download processed images, masks, videos and data.
8. To build a polished, responsive web interface and verify the system with automated tests.

2.3 Limitations of Existing Approaches

Table 2.1: *Existing approaches and their drawbacks*

Approach	Drawbacks
Manual inspection and counting	Slow, subjective, impossible for long videos
Fixed RGB thresholding scripts	Sensitive to shadows and highlights; no interface; no measurements or analytics
Image editors with colour pickers	Can select a colour but cannot count objects, export data or track motion
Deep-learning object detectors	Need labelled data, training and often a GPU; hard to explain; excessive when colour alone identifies the object

3. METHODOLOGY

3.1 Overall Approach

The project follows a classical computer-vision pipeline. The same pipeline is applied to a single image or, for a video, to every frame in turn. The development methodology consisted of the following steps:

1. **Requirement study** – identification of the colour detection, tracking and analytics features required.
2. **Colour model selection** – HSV was chosen over RGB because hue carries the colour while shading mainly affects the value (brightness) channel.
3. **Threshold design** – HSV ranges were defined for six colours on the OpenCV scale (H: 0–179, S and V: 0–255) and exposed through sliders for fine-tuning.
4. **Noise handling** – morphological opening followed by closing with an elliptical kernel; contours smaller than a minimum area are discarded.
5. **Object description** – bounding rectangle, contour area and centroid (from image moments) are computed for every contour.
6. **Tracking design** – greedy nearest-centroid matching with a maximum match distance and a disappearance counter.
7. **Interface and analytics** – a five-page Streamlit application in which every chart reads directly from the stored results.
8. **Testing** – a synthetic dataset with exact ground truth, automated unit tests (pytest) and automated browser tests (Playwright).

3.2 Image Processing Techniques Used

Table 3.1: *Image processing techniques implemented*

No.	Technique	Purpose (in simple words)	OpenCV function
1	Image loading	Decode uploaded bytes into a BGR pixel array; very large images are scaled to 1600 px	cv2.imdecode, cv2.resize
2	BGR to HSV conversion	Separate colour (hue) from colour strength (saturation) and brightness (value)	cv2.cvtColor
3	HSV colour segmentation	Describe a colour as a box in HSV space: hue interval plus minimum S and V	–
4	Thresholding	Pixels inside the HSV box become 255 (white), all others 0 (black)	cv2.inRange

No.	Technique	Purpose (in simple words)	OpenCV function
5	Binary masking	Keep only the target-colour pixels of the original image	cv2.bitwise_and
6	Erosion	Shrink white regions – removes isolated noise pixels	cv2.erode
7	Dilation	Grow white regions – fills small gaps	cv2.dilate
8	Morphological opening	Erosion then dilation – removes noise while keeping object size	cv2.morphologyEx (OPEN)
9	Morphological closing	Dilation then erosion – fills small holes such as highlights	cv2.morphologyEx (CLOSE)
10	Contour detection	Trace the outline of each connected white region	cv2.findContours
11	Bounding box	Smallest upright rectangle enclosing a contour	cv2.boundingRect
12	Centroid calculation	Centre of mass: cx = M10/M00, cy = M01/M00	cv2.moments
13	Frame-by-frame processing	Read, process and write one frame at a time (constant memory)	cv2.VideoCapture / VideoWriter
14	Centroid-based tracking	Match centroids between frames by smallest Euclidean distance	CentroidTracker (own code)

3.3 HSV Colour Model and the Red Hue Problem

In OpenCV the hue channel is stored from 0 to 179 (the 360° colour wheel divided by two). Red lies exactly where the wheel wraps around, i.e. close to both 0 and 179. A single interval such as [170, 10] cannot be expressed in one cv2.inRange() call, so red is detected as the union of two ranges, [0, 10] OR [170, 179], whose masks are merged with cv2.bitwise_or(). A unit test confirms that using only the lower range finds just one of two red objects, while the dual range finds both.

Table 3.2: *Default HSV ranges for the predefined colours (adjustable in the application)*

Colour	Hue range	Min. saturation	Min. value
Red	0–10 and 170–179	120	70
Orange	11–20	120	100
Yellow	21–34	100	100
Green	35–85	60	50
Blue	95–128	80	50
Purple	129–160	60	50

3.4 Algorithm 1 – Colour Object Detection

```
Input : image I (BGR), list of HSV ranges R, kernel size k, minimum area A
Output: list of objects (box, area, centroid), annotated image

1. HSV <- cvtColor(I, BGR2HSV)
2. M   <- empty (all-zero) mask of the size of I
3. for each (lower, upper) in R:          # 2 ranges for red, 1 otherwise
       M <- M OR inRange(HSV, lower, upper)
4. M   <- morphologyEx(M, OPEN,  ellipse(k))   # remove noise
5. M   <- morphologyEx(M, CLOSE, ellipse(k))   # fill holes
6. C   <- findContours(M, RETR_EXTERNAL)
7. for each contour c in C with contourArea(c) >= A:
       (x, y, w, h) <- boundingRect(c)
       m <- moments(c);  cx <- m10/m00;  cy <- m01/m00
       store object, draw box, label and centroid on I
8. return objects (sorted by area), I
```

3.5 Algorithm 2 – Centroid-Based Tracking (for each frame t)

```
Input : detections D_t = {(centroid, box)} of frame t
State : tracks T = {id -> (centroid, box, missing)}, next_id
Params: max_distance d_max, max_disappeared m_max

1. if D_t is empty: missing += 1 for every track; delete tracks with missing
> m_max
2. if T is empty: create a new track (next_id++) for every detection
3. Dist[i][j] <- Euclidean distance between track i and detection j
4. for all pairs (i, j) in increasing order of Dist:
       skip if track i or detection j is already matched
       stop if Dist[i][j] > d_max
       update track i with detection j; missing_i <- 0
5. unmatched tracks: missing += 1; delete if missing > m_max
6. unmatched detections: start a new track with next_id
7. draw box, 'ID n' label and the trail of recent centroids
```

The disappearance counter (step 5) is what allows an object to keep its ID when it is hidden for a few frames, for example behind an obstacle. The matching is greedy (closest pair first) rather than an optimal assignment; for a small number of well-separated objects this gives the same result with much simpler code. The tracker uses position only – it is not a deep-learning or Kalman-filter tracker.

4. SYSTEM ARCHITECTURE

4.1 Architecture of the Processing Model

Chromatic Vision does not use a trained machine-learning model. Its "model" is the deterministic image-processing pipeline of Chapter 3, organised into a three-layer software architecture (Figure 4.1):

- **Presentation layer (Streamlit)** – app.py (page configuration and routing), reusable UI components, one module per page and a custom stylesheet.
- **Processing layer (modules/)** – all computer-vision logic: colour ranges, image processing, video processing, object tracking and analytics. This layer has no dependency on the user interface, which makes it easy to test.
- **Foundation layer** – the OpenCV, NumPy, Plotly and Matplotlib libraries, Streamlit session state (results and statistics) and a per-session folder for temporary files that is cleaned automatically.

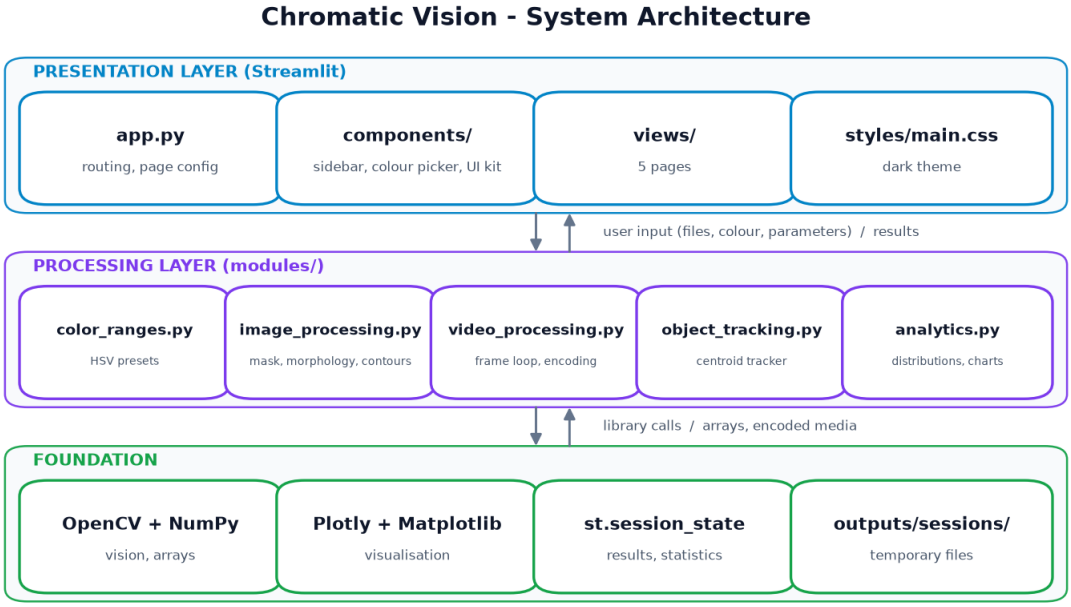


Figure 4.1: *System architecture of Chromatic Vision*

4.2 Processing Workflow

Figure 4.2 shows the flow of data through the processing model. For a still image steps 1–6 and 8 are executed once. For a video, steps 2–7 are repeated for every frame and each annotated frame is written to the output video.

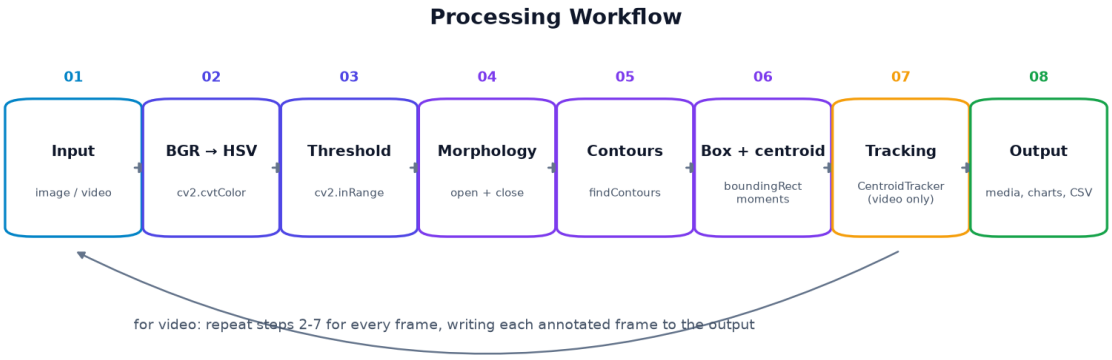


Figure 4.2: *Processing workflow (steps 2–7 are repeated for every video frame)*

4.3 Technologies Used

Table 4.1: *Technology stack*

Technology	Role in the project
Python 3.10+	Implementation language (developed and tested on Python 3.12.10)
OpenCV	Image decoding, colour conversion, thresholding, morphology, contours, moments, video input/output
NumPy	Pixel arrays, distance matrix for tracking, statistics
Streamlit	Web interface, widgets, session state, file upload and download
Plotly	Interactive charts: colour distribution, object areas, per-frame charts, trajectories
Matplotlib	Hue-spectrum histogram and report figures
Pillow	Reading file format information of uploaded images; image display in Streamlit
pytest, Playwright	Automated unit tests and automated browser (end-to-end) tests

4.4 Hardware and Software Requirements

Table 4.2: *Hardware and software requirements*

Item	Minimum requirement	System used for testing
Processor	Dual-core 64-bit CPU	AMD Ryzen 5 5600G (6 cores / 12 threads)
RAM	4 GB	16 GB
GPU	Not required	Not used
Operating system	Windows 10/11, Linux or macOS	Windows 10 Pro
Software	Python 3.10+, modern web browser	Python 3.12.10, Microsoft Edge

5. DATASET DETAILS

5.1 Description of the Dataset

Because the system is based on image processing rather than machine learning, it does not require a training dataset. A dataset is nevertheless needed to **evaluate** the detection and tracking accuracy. For this purpose a small sample dataset was created synthetically with OpenCV and NumPy (scripts/generate_samples.py). Generating the data has two important advantages: the exact number and position of objects of every colour is known (ground truth), and there are no copyright or licensing restrictions.

To make the data realistic, the scenes contain shaded balls with highlights (constant hue but varying brightness), flat geometric shapes, neutral distractor objects (white, grey and black) that must not be detected, a gradient background and added Gaussian noise. The ground truth is stored in assets/sample_ground_truth.json.

Table 5.1: *Sample dataset*

File	Type	Resolution	Length	Content
shapes_scene.png	Image (PNG)	1280 × 800	–	15 coloured shapes (circles, rectangles, triangles) in all six colours + 3 neutral distractors
ball_collection.jpg	Image (JPEG)	1000 × 640	–	9 shaded balls: 4 red, 3 blue, 2 green
neutral_scene.png	Image (PNG)	900 × 600	–	Only white, grey and black objects (negative test)
moving_objects.mp4	Video (MP4)	640 × 360	200 frames, 25 fps, 8 s	2 red balls in opposite lanes, 1 bouncing blue ball, 1 green square, 1 yellow ball on a sine path, and a grey pillar that hides one red ball
orbiting_ball.avi	Video (AVI, MJPG)	480 × 360	120 frames, 30 fps, 4 s	1 orange ball moving on a circular path

Table 5.2: *Ground truth – number of objects of each colour*

File	Red	Orange	Yellow	Green	Blue	Purple
shapes_scene.png	3	2	2	3	3	2
ball_collection.jpg	4	0	0	2	3	0
neutral_scene.png	0	0	0	0	0	0
moving_objects.mp4	2	0	1	1	1	0
orbiting_ball.avi	0	1	0	0	0	0

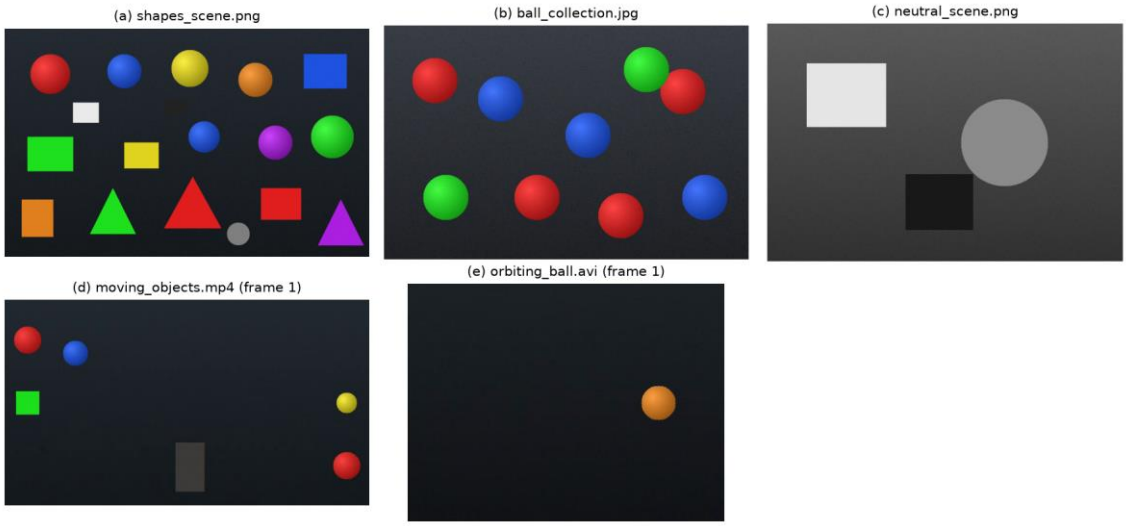


Figure 5.1: *Sample dataset: (a)–(c) images, (d)–(e) first frames of the videos*

5.2 User Data

In addition to the samples, the user can upload any JPG, JPEG or PNG image and any MP4, AVI or MOV video (up to 200 MB). Images larger than 1600 pixels on the longest side and videos wider than the chosen processing width are scaled down automatically to keep memory use and processing time reasonable.

6. IMPLEMENTATION

6.1 Project Structure

The source code is modular: the computer-vision logic is completely separated from the user interface.

Table 6.1: *Main source files*

File / folder	Responsibility
app.py	Entry point: page configuration, loading of the stylesheet, page routing
modules/color_ranges.py	HSV presets for six colours, dual-range builder for red, colour conversions
modules/image_processing.py	load_image_from_bytes, to_hsv, create_mask, apply_morphology, find_objects, draw_detections, analyze_image
modules/object_tracking.py	CentroidTracker class with update(), track history and summary
modules/video_processing.py	get_video_info, make_preview, process_video (frame loop, overlays, WebM and MP4 writers)
modules/analytics.py	Colour distribution and hue histogram; Plotly and Matplotlib chart builders
components/	Session state and navigation, sidebar, colour picker, reusable HTML/CSS components
views/	One module per page: Overview, Image Analysis, Video Tracking, Analytics, About
styles/main.css	Custom dark theme (#0B1020 background, indigo / sky-blue / violet accents)
tests/	34 automated unit tests (pytest)
scripts/	Sample generation, report figures, automated browser test and screenshots

6.2 Image Analysis Module

analyze_image() runs the complete pipeline of Algorithm 1 and returns a result object containing the original image, HSV image, raw mask, every morphological stage, the final mask, the segmented image, the annotated image, the list of objects, the colour coverage percentage and the processing time. The objects are sorted by area and labelled “Red #1”, “Red #2”, etc. on the annotated image, with a white dot marking the centroid.

6.3 Video Tracking Module

process_video() opens the input with cv2.VideoCapture and reads one frame at a time, so memory use stays constant regardless of video length. Each frame is optionally resized, processed with the detection pipeline and passed to the CentroidTracker. The tracked objects are drawn with a colour per ID, an "ID n" label and a motion trail of recent centroids, together with a status bar (colour, frame number, object count). Each frame is written to two output files:

- **WebM (VP8)** – plays directly in every modern browser and is used for the in-app player;
- **MP4 (MPEG-4 Part 2)** – for desktop players such as VLC and Windows Media Player. (The OpenCV build for Windows does not include an H.264 encoder, which was confirmed during development.)

Per-frame statistics (number of objects, active IDs and processing time) are recorded for the analytics page and the CSV download. A progress callback updates the progress bar and a live preview in the browser while the video is processed.

6.4 User Interface

The interface is a Streamlit application with a custom dark theme and five pages selected from the sidebar: **Overview** (hero section, live session statistics, quick demos and feature cards), **Image Analysis Studio**, **Video Tracking Studio**, **Analytics & Insights** and **About Project** (in-app documentation). Streamlit session state stores the current page, the statistics, the latest results and the run history. The layout adapts to phone screens.

6.5 Error Handling and Resource Management

- Empty, corrupt or non-image files raise a ValueError that the interface shows as a clear error message instead of crashing.
- Unreadable or empty videos are detected before processing; videos with no matching colour are processed normally and report zero detections.
- Uploads, previews and generated videos are stored in a per-session folder; old outputs are deleted when replaced, and folders older than 12 hours are removed automatically.

6.6 Testing Method

Two types of automated tests were written and executed. **Unit tests** (pytest, 34 tests) check the processing layer against the known ground truth, including edge cases such as invalid

files, empty videos, very large inputs and tracker behaviour during occlusions. **End-to-end tests** (Playwright with Microsoft Edge) operate the real application in a browser exactly as a user would – navigating every page, running both demos, changing colours, uploading valid and invalid files and downloading the results – and also capture the screenshots used in this report.

7. RESULTS AND DISCUSSION

All values in this chapter were obtained from actual runs of the system on the test machine described in Table 4.2. Processing times vary slightly from run to run.

7.1 Image Detection Results

Table 7.1: *Image detection results with default settings (kernel 5 × 5, 1 iteration, minimum area 300 px)*

Image	Colour	Expected	Detected	Coverage (%)	Time (ms)
shapes_scene.png	Red	3	3	4.81	29.5 *
shapes_scene.png	Orange	2	2	2.52	7.5
shapes_scene.png	Yellow	2	2	2.37	8.5
shapes_scene.png	Green	3	3	4.90	6.6
shapes_scene.png	Blue	3	3	3.83	7.6
shapes_scene.png	Purple	2	2	2.37	9.5
ball_collection.jpg	Red	4	4	7.63	5.6
ball_collection.jpg	Blue	3	3	5.93	4.5
ball_collection.jpg	Green	2	2	3.91	4.7
neutral_scene.png	All six	0	0	0.00	–

* The first call includes a one-time OpenCV initialisation cost; subsequent calls on the same image take 7–10 ms.

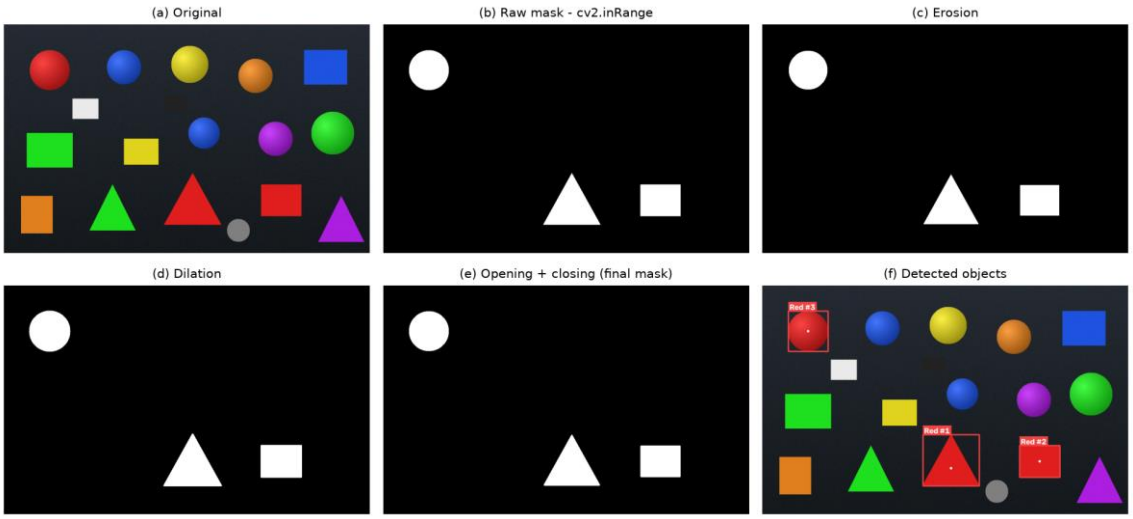


Figure 7.1: *Pipeline stages for red on shapes_scene.png: (a) original, (b) raw mask, (c) erosion, (d) dilation, (e) final mask, (f) detected objects*

Discussion. The detected count equals the ground truth for every colour on both sample images (15 of 15 colour/image combinations). The neutral distractors are never detected, because the minimum saturation and value thresholds exclude achromatic pixels – something a simple RGB threshold cannot guarantee. The shaded red ball, whose brightness varies strongly from its highlight to its edge, is detected as a single object, which demonstrates the advantage of the HSV colour space.

7.2 Video Tracking Results

Table 7.2: *Video tracking results (max. match distance 80 px, 15 frames to keep a lost ID)*

Video	Colour	Frames	Frames with detections	Detection freq.	Unique IDs	Avg. ms / frame
moving_objects.mp4	Red	200	200	100 %	2	9.44
moving_objects.mp4	Blue	200	200	100 %	1	8.73
moving_objects.mp4	Purple (absent)	200	0	0 %	0	8.22
orbiting_ball.avi	Orange	120	120	100 %	1	9.97

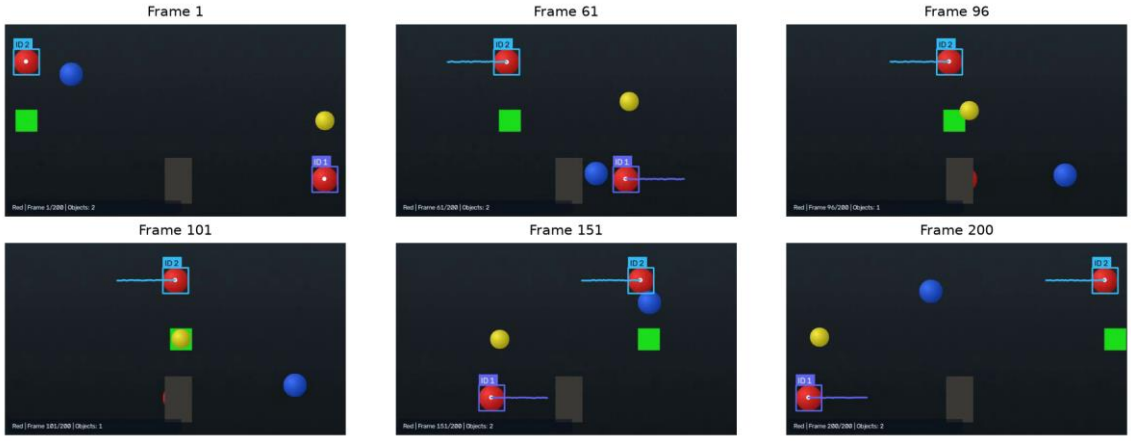


Figure 7.2: *Frames 1, 61, 96, 101, 151 and 200 of the processed sample video. In frames 96 and 101 the lower red ball (ID 1) is hidden behind the pillar; after reappearing it keeps ID 1*

Discussion. The tracker produced exactly **two IDs** for the two red balls over the whole clip. Track 1 was matched in 191 of the 200 frames: for 9 frames it was hidden behind the pillar, and the disappearance counter kept the track alive so that the ball received the same ID when it reappeared. Track 2 was matched in all 200 frames. When the selected colour is absent (purple) the system runs without error, reports zero detections and still produces an output video. An average of about 9–10 ms per frame corresponds to roughly 100 frames

per second, which is faster than real time for these videos. The output WebM and MP4 files were re-opened with OpenCV and contained the expected number of frames.

7.3 Summary of Test Cases

Table 7.3: *Test cases and results*

ID	Test	Expected result	Actual result	Status
TC-01	Red detection (both hue ends; sample image)	Both ends found; 3 red objects	2 of 2 found; 3 red objects	Pass
TC-02	Blue detection	1 object, centroid at centre (±2 px)	As expected	Pass
TC-03	Green detection	Exact bounding box; 2 green balls in sample	As expected	Pass
TC-04	Multiple objects, all colours on all samples	Counts equal ground truth	15 of 15 match	Pass
TC-05	No matching colour	0 objects, info message, no crash	As expected	Pass
TC-06	Invalid image upload (text file renamed .png)	Clear error message	“Invalid image” shown	Pass
TC-07	Video processing (MP4, AVI, MOV)	Correct video info; processed	As expected	Pass
TC-08	Moving object tracking with occlusion	2 IDs for 2 red balls	2 IDs, kept through occlusion	Pass
TC-09	Processed image download	Valid PNG file	PNG downloaded and decoded	Pass
TC-10	Processed video download	WebM and MP4 open with all frames	As expected	Pass
TC-11	Dashboard statistics	Counts update after real runs only	5 images, 1 video shown	Pass
TC-12	Navigation between pages	All 5 pages open	As expected	Pass
TC-13	Responsive layout (390 px)	No horizontal scrolling	As expected	Pass

Overall, all **34 unit tests** and all **23 automated browser checks** passed.

8. SCREENSHOTS

The following screenshots were captured from the running application with the automated browser test.

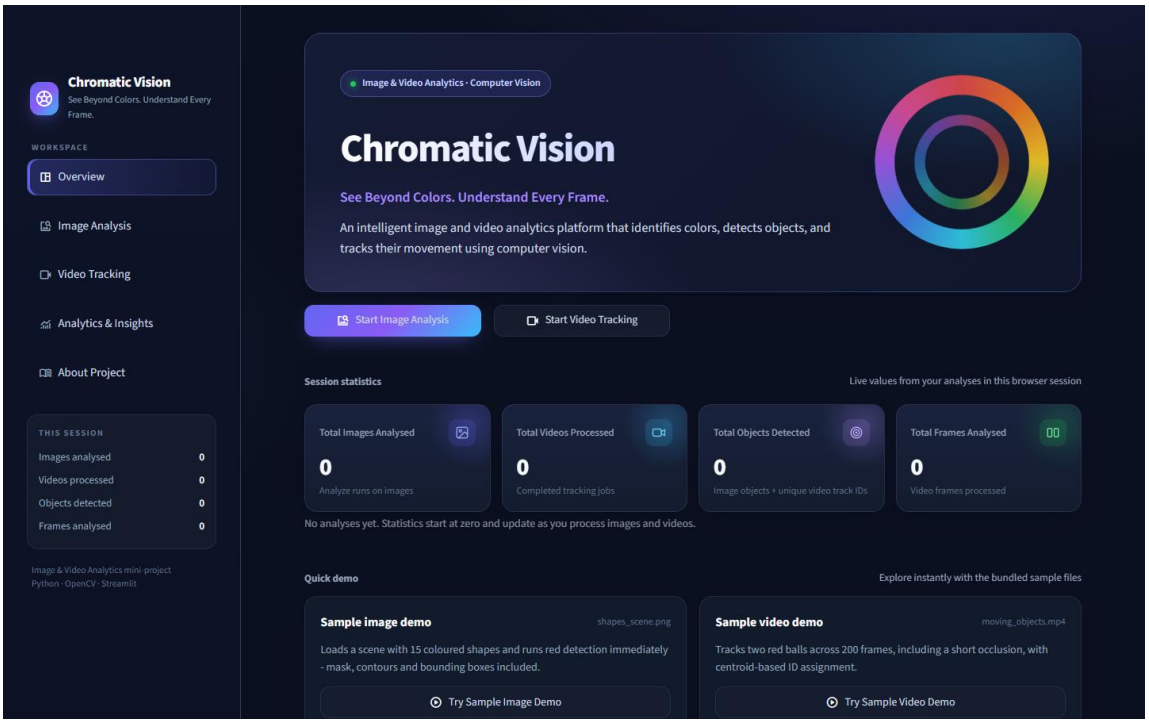


Figure 8.1: *Overview dashboard with live session statistics and navigation sidebar*

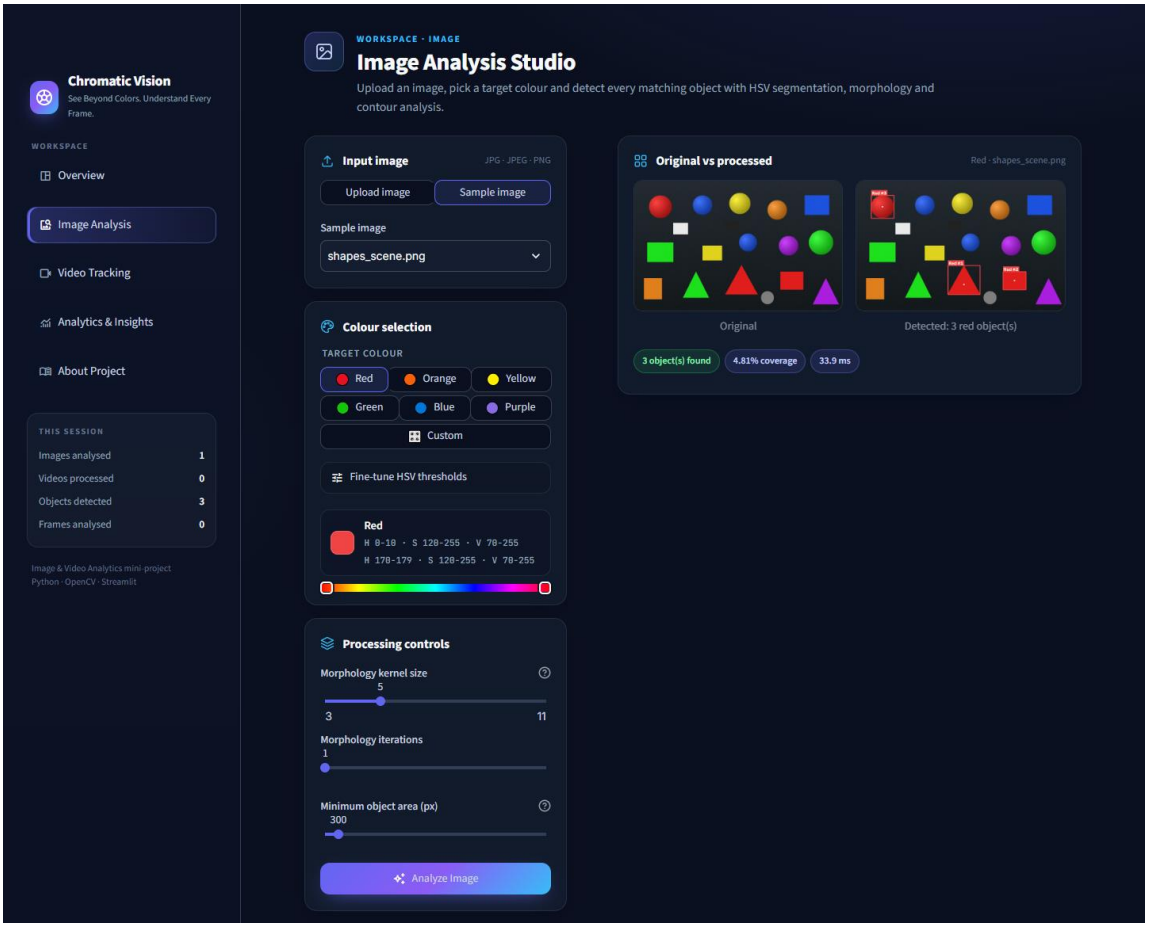


Figure 8.2: *Image Analysis Studio – input, colour palette, HSV range and original vs. processed comparison*

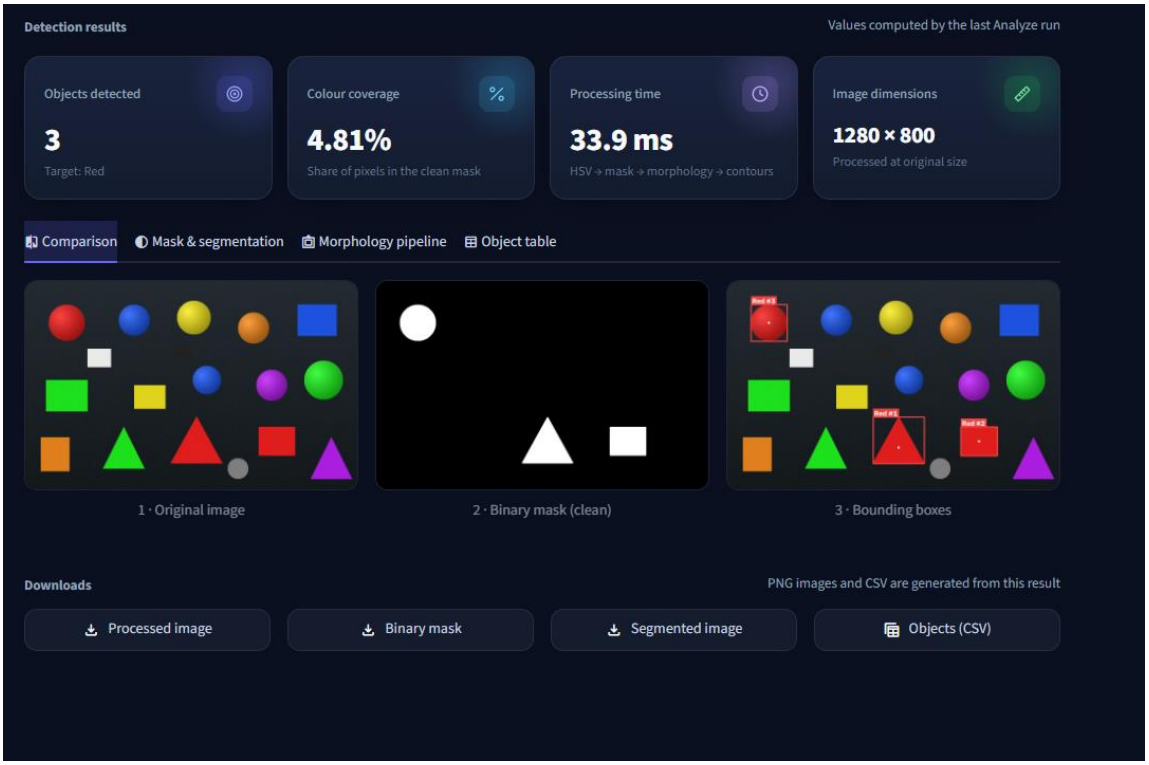


Figure 8.3: *Image detection results – metrics, comparison of original, mask and bounding boxes, downloads*

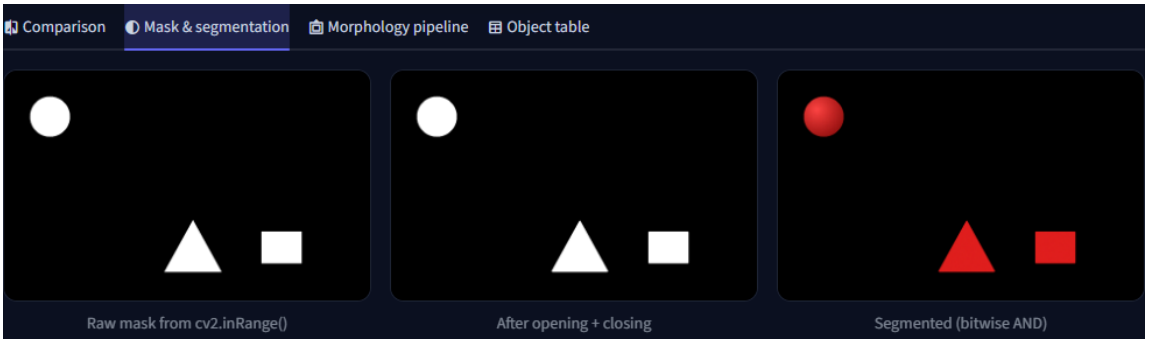


Figure 8.4: *Mask and segmentation tab – raw mask, cleaned mask and segmented image*

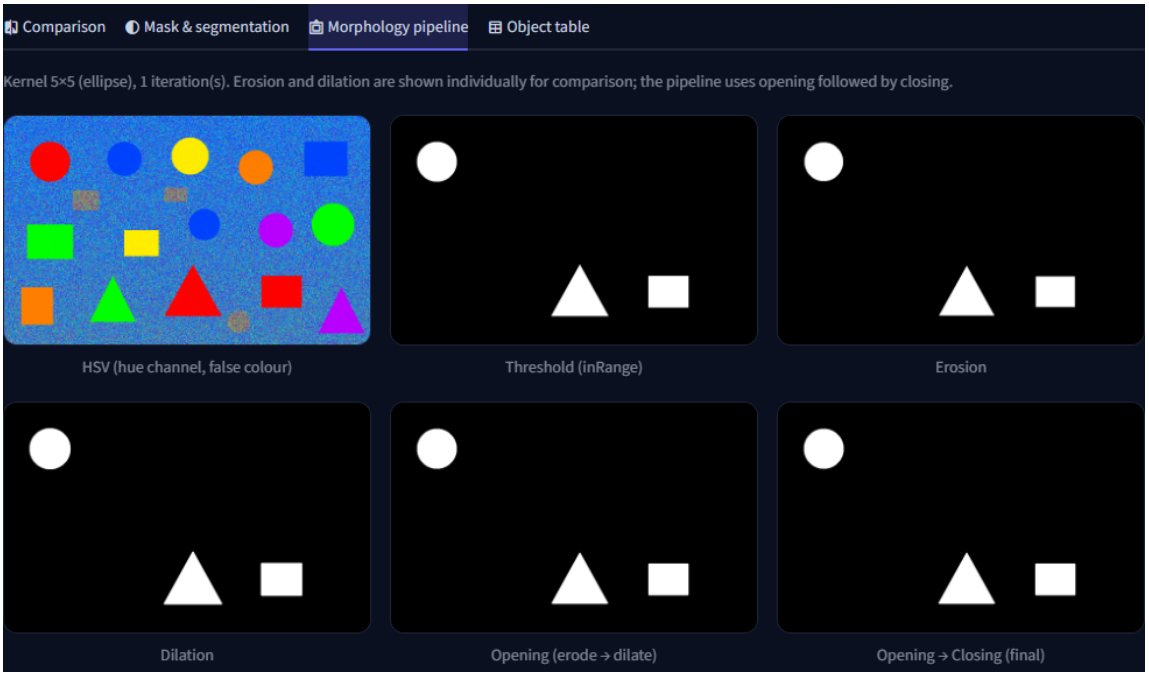


Figure 8.5: *Morphology pipeline tab – HSV hue, threshold, erosion, dilation, opening and closing*

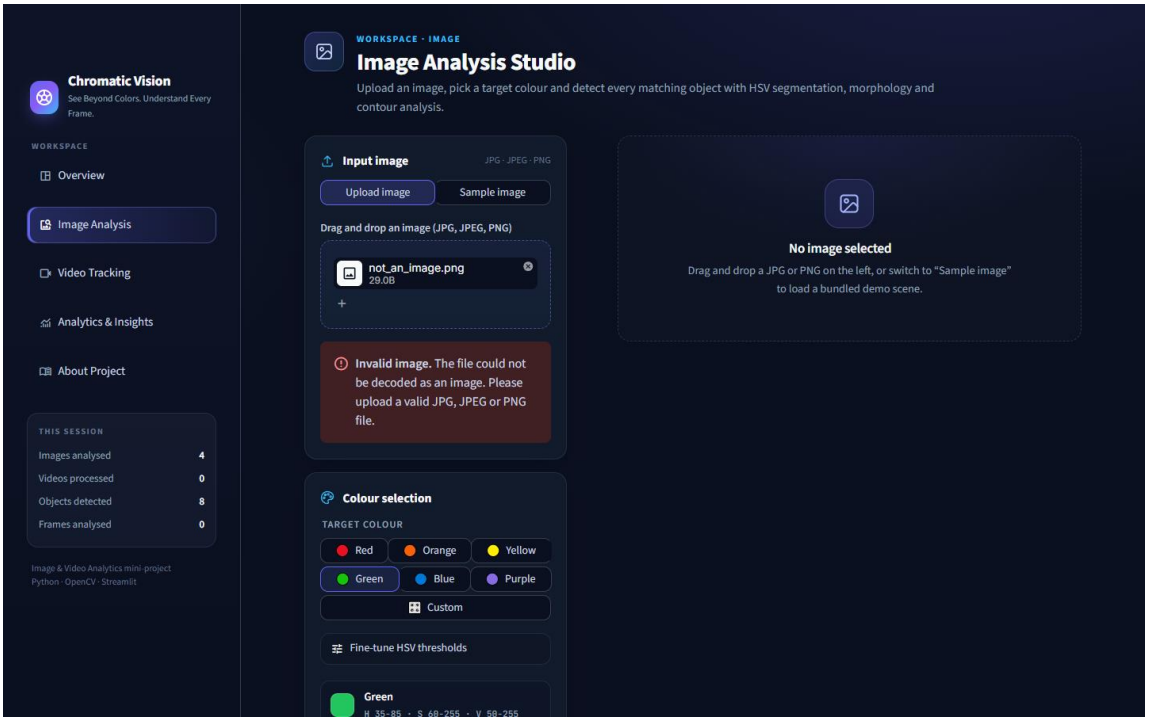


Figure 8.6: *Error handling – a corrupt file uploaded as an image is rejected with a clear message*

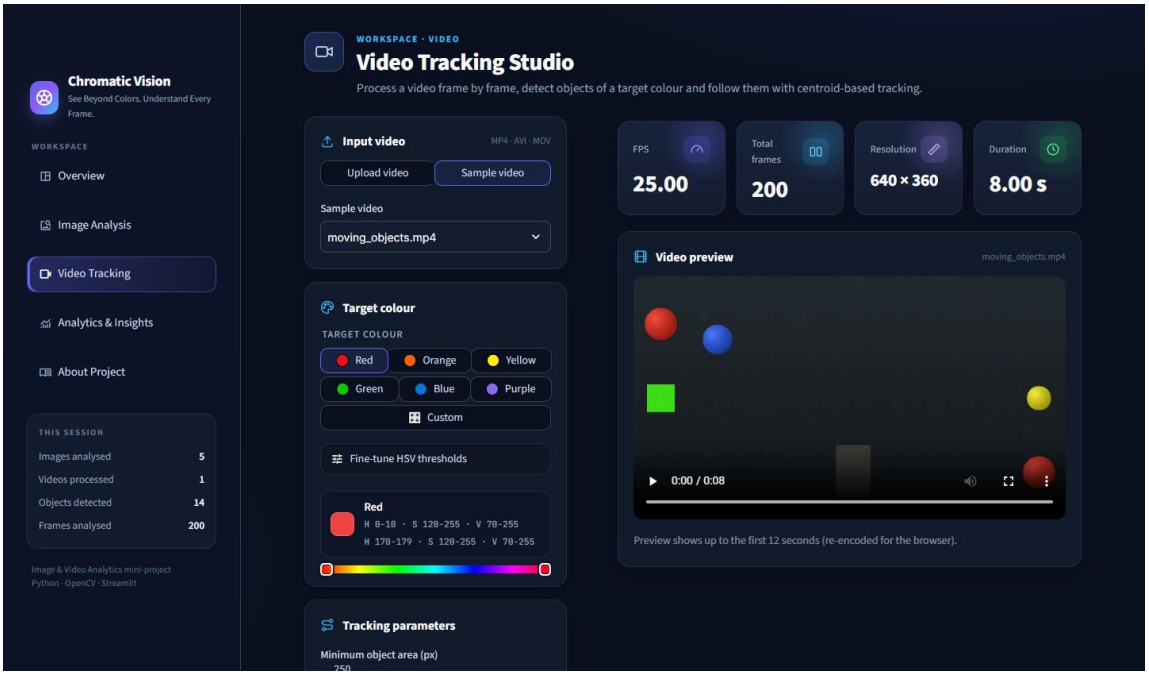


Figure 8.7: *Video Tracking Studio – video information (FPS, frames, resolution, duration) and preview*

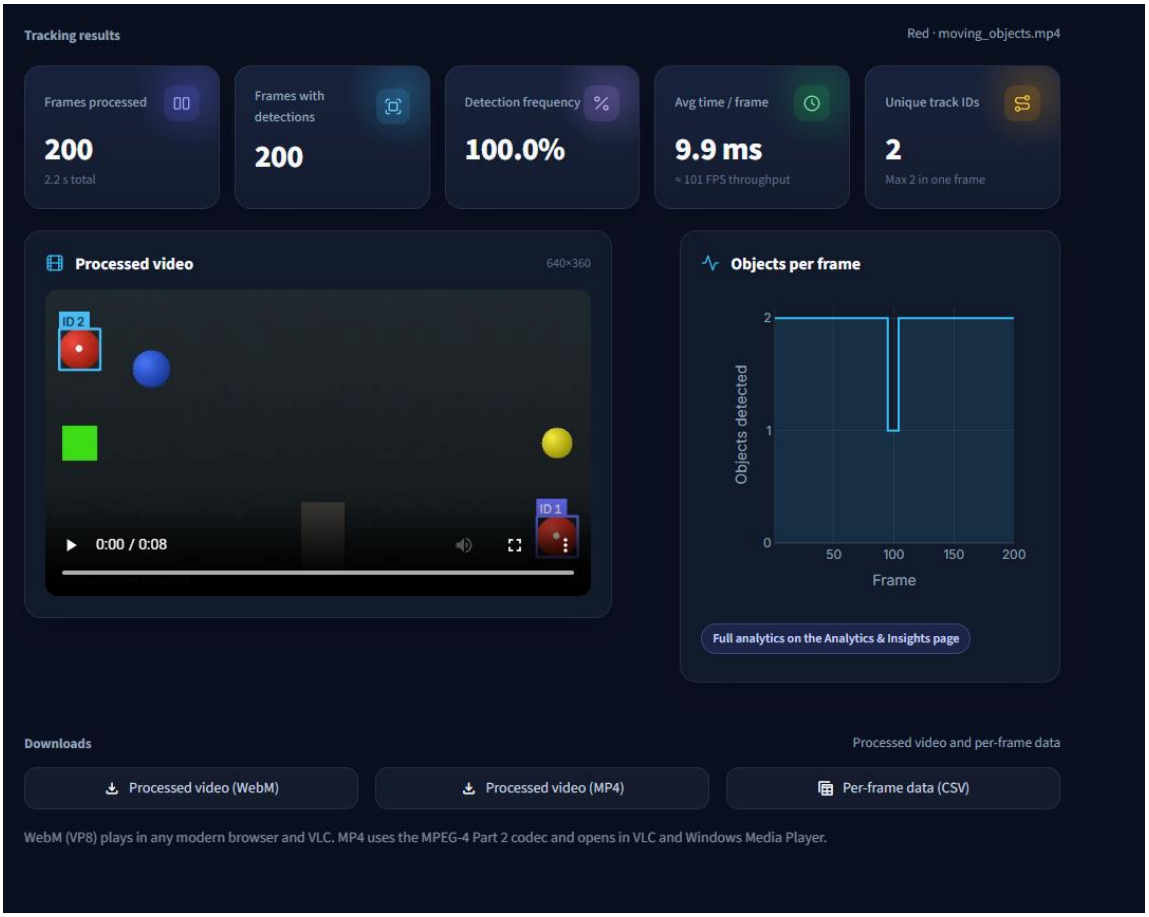


Figure 8.8: *Video tracking results – metrics, processed video with IDs and trails, downloads*

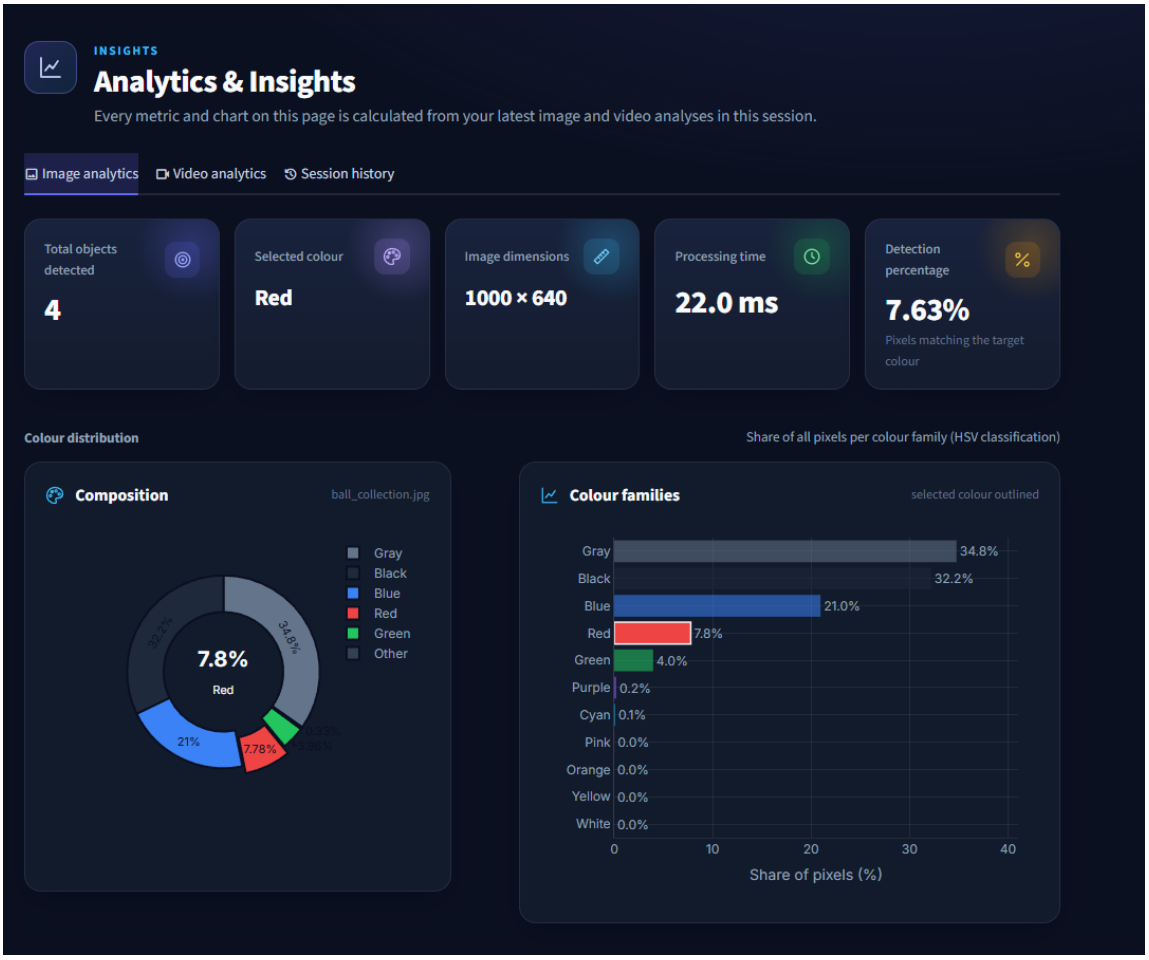


Figure 8.9: *Analytics – image metrics and colour distribution charts*



Figure 8.10: *Analytics – object areas, Matplotlib hue spectrum and visual comparison*

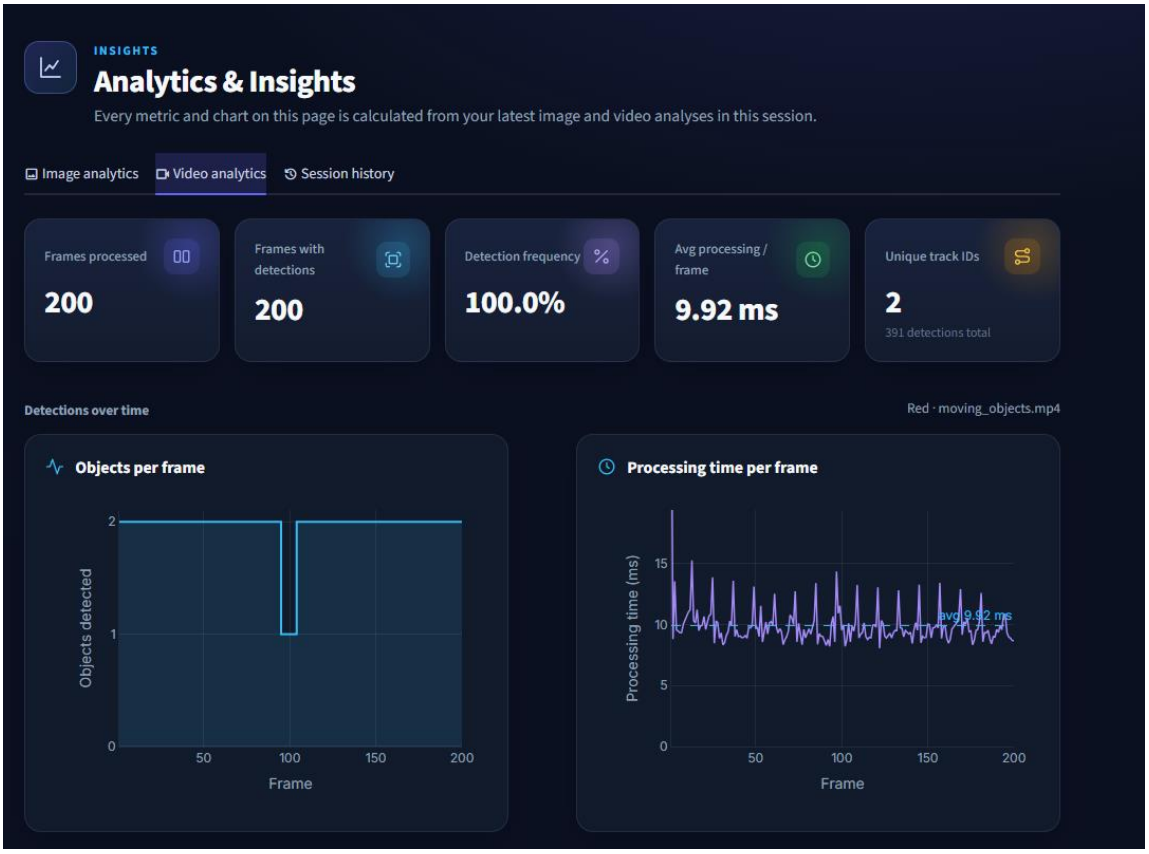


Figure 8.11: *Analytics – video metrics, objects per frame and processing time per frame*

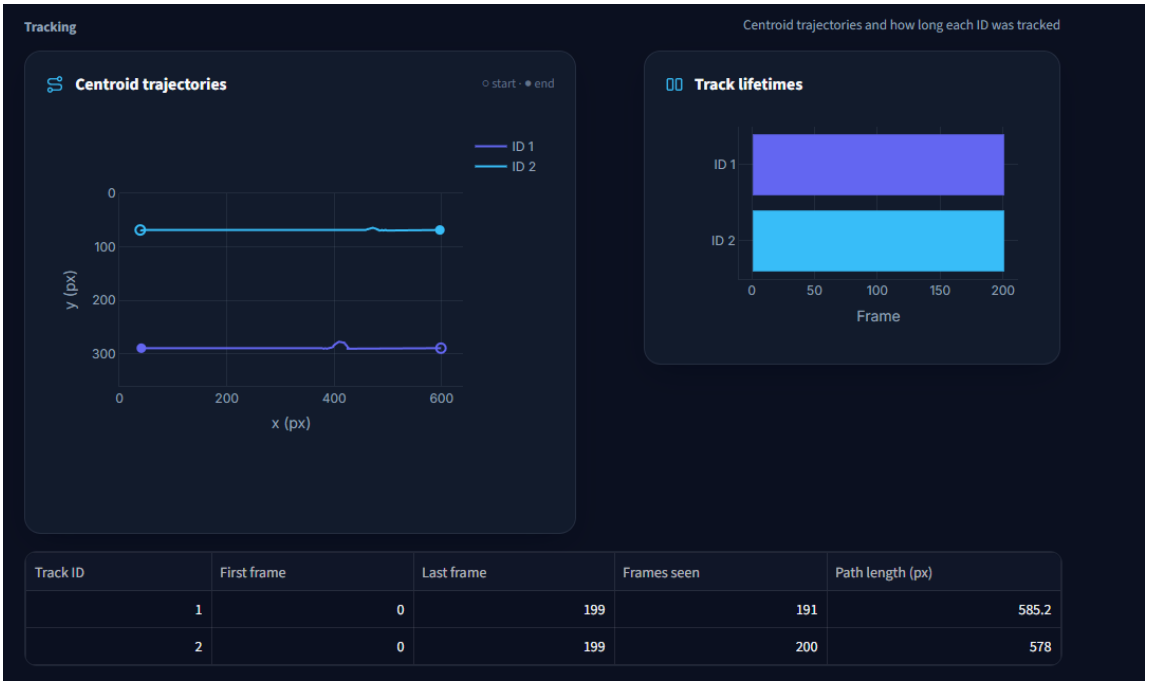


Figure 8.12: *Analytics – centroid trajectories and track lifetimes*

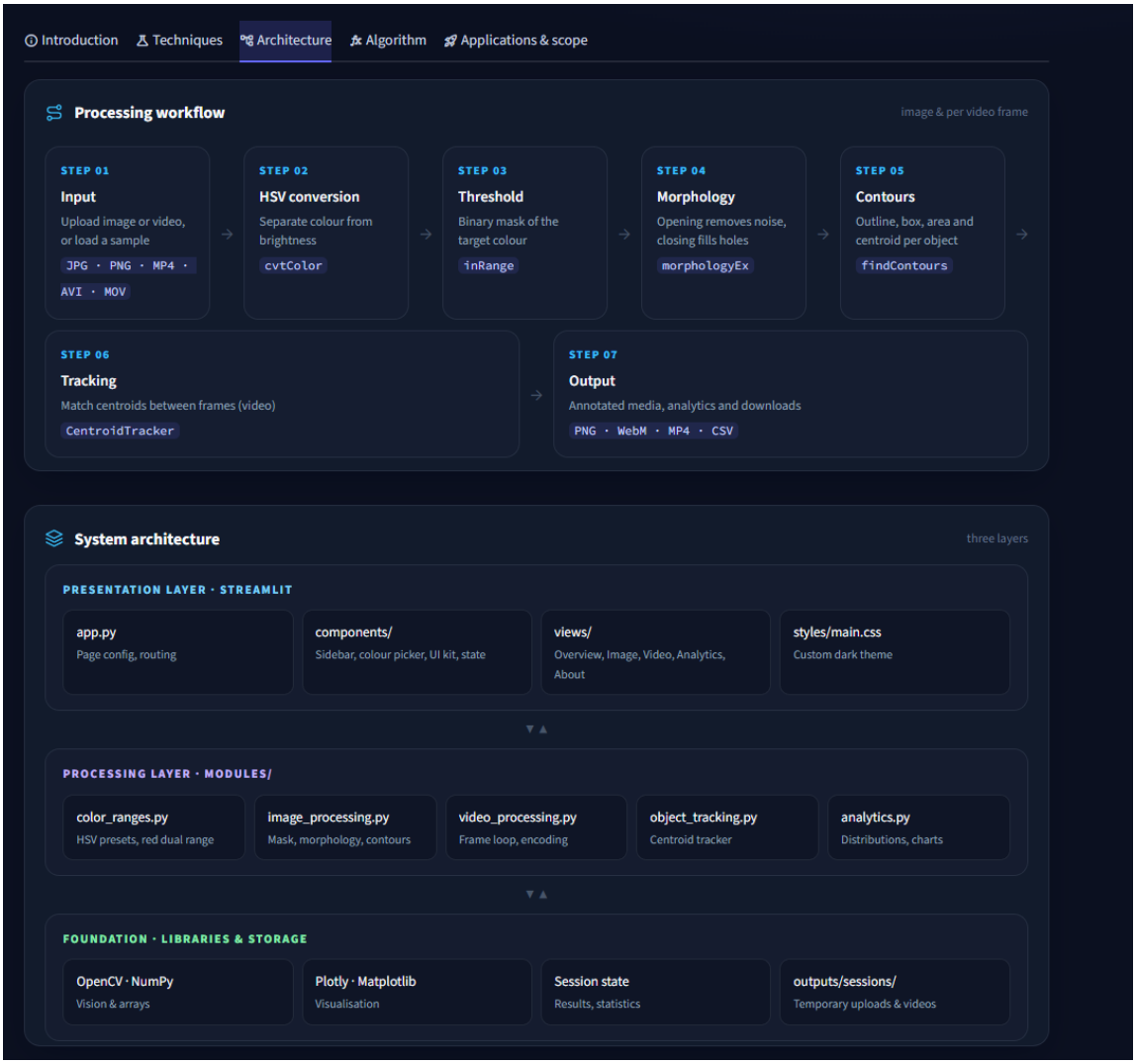


Figure 8.13: *About Project page – in-app workflow and architecture diagrams*

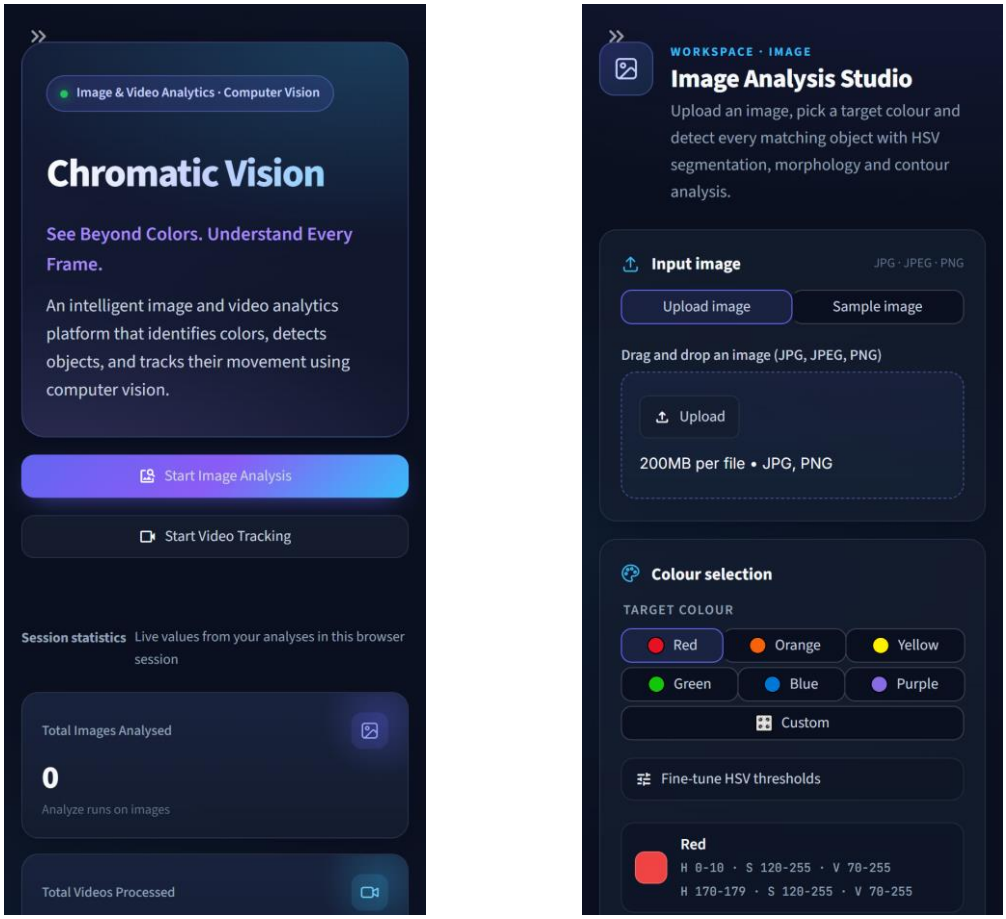


Figure 8.14: *Responsive layout on a phone screen (390 px wide): Overview and Image Analysis*

9. CONCLUSION

9.1 Conclusion

This project shows that a classical image processing pipeline – HSV colour conversion, cv2.inRange() thresholding, morphological cleaning, contour analysis and centroid tracking – is sufficient to detect, measure and track colour-defined objects reliably and faster than real time, without any training data or GPU. All the objectives stated in Chapter 2 were achieved: six colour presets plus custom ranges, correct dual-range red detection, configurable morphology, frame-by-frame video tracking with consistent IDs through short occlusions, analytics computed from real results, downloadable outputs and a polished, responsive web interface.

On the evaluation dataset, the detected object counts matched the ground truth exactly for every colour, neutral objects were never falsely detected, and both red balls kept their identities for all 200 frames of the sample video. All 34 unit tests and 23 automated browser checks passed.

9.2 Limitations

- Fixed HSV thresholds are sensitive to strong colour casts and very dim lighting.
- Touching objects of the same colour merge into one contour, and background regions of the target colour are detected as objects.
- Centroid matching uses position only, so IDs may swap when two same-coloured objects cross very closely.
- The evaluation dataset is synthetic; accuracy on real photographs depends on threshold tuning and was not measured on a labelled real-world dataset.

9.3 Future Enhancements

- Live webcam processing.
- Kalman-filter motion prediction and Hungarian assignment for more robust tracking.
- Watershed segmentation to separate touching objects.
- Automatic threshold calibration from a pixel clicked by the user.
- Shape classification (circle, rectangle, triangle) and multi-colour tracking in one pass.

REFERENCES

1. R. C. Gonzalez and R. E. Woods, Digital Image Processing, 4th ed. Pearson, 2018.
2. G. Bradski and A. Kaehler, Learning OpenCV: Computer Vision with the OpenCV Library. O'Reilly Media, 2008.
3. R. Szeliski, Computer Vision: Algorithms and Applications, 2nd ed. Springer, 2022.
4. A. R. Smith, "Color gamut transform pairs," ACM SIGGRAPH Computer Graphics, vol. 12, no. 3, pp. 12–19, 1978.
5. J. Serra, Image Analysis and Mathematical Morphology. Academic Press, 1982.
6. S. Suzuki and K. Abe, "Topological structural analysis of digitized binary images by border following," Computer Vision, Graphics, and Image Processing, vol. 30, no. 1, pp. 32–46, 1985.
7. OpenCV documentation – Changing Colorspaces, Morphological Transformations, Contours. https://docs.opencv.org/
8. Streamlit documentation. https://docs.streamlit.io/
9. Plotly Python documentation. https://plotly.com/python/